IoT Development Team - Car Automation Internship

DATE: 1/07/2026

ELM327:

ELM327 is a widely popular, low-cost microcontroller programmed to translate the on-board diagnostics (OBD) interface found in most modern cars. It acts as a bridge between your vehicle's computer (ECU) and a device like a smartphone, tablet, or laptop, allowing you to read trouble codes, view real-time engine data, and clear check engine lights.

Here is a breakdown of what it is, how it works, and how to use it:

1. How it Works

Most modern cars (built after 1996 in the US, 2001 in Europe, and 2010 in India) feature an **OBD-II port** usually located under the dashboard near the driver's side.

- The ELM327 adapter plugs directly into this port.
- The chip inside protocols the standard automotive languages (like CAN bus, ISO 9141, J1850) into a standard serial stream (AT commands) that consumer electronics can understand.
- It transmits this data to your device via **Bluetooth, Wi-Fi, or a USB cable**.

2. Common Use Cases

- **Read Diagnostic Trouble Codes (DTCs):** When your "Check Engine" light comes on, the ELM327 can tell you the exact code (e.g., P0420 for a catalytic converter issue).
- **Clear the Check Engine Light:** Once you know the issue (or have fixed it), you can turn the warning light off.
- **Real-Time Data Monitoring:** You can view live data while driving, such as RPM, vehicle speed, engine coolant temperature, fuel trim, oxygen sensor voltage, and mass airflow (MAF) readings.
- **Performance Tracking:** Many apps use the data to calculate 0-60 mph times, fuel efficiency (MPG or km/l), and horsepower.

3. Connection Types

When buying an ELM327 scanner, you will generally find three versions:

- **Bluetooth (Most Common):** Great for Android devices and Windows laptops. (Note: Older Bluetooth 3.0 adapters don't always work with iOS; look for **Bluetooth 4.0/LE** if you use an iPhone).
- **Wi-Fi:** The preferred choice for iOS (iPhone/iPad) users, though it works with Android as well. It creates its own mini Wi-Fi network that you connect your phone to.
- **USB:** Best for stable, physical connections to a laptop/PC for deeper diagnostics or software flashing.

4. Popular Compatible Apps

The ELM327 hardware requires software to visualize the data. Some of the most popular apps include:

- **Torque Pro / Lite (Android):** Highly customizable dashboards and powerful logging.
- **Car Scanner ELM OBD2 (Android, iOS, Windows):** Excellent, clean UI with great compatibility for modern cars.
- **OBD Fusion (Android, iOS):** A robust option with professional-grade diagnostics.
- **DashCommand (Android, iOS):** Great for visual gauges and performance tracking.

Here is a breakdown of how each component and data point works, where the data comes from, and how it is collected.

1. Engine & Drivetrain Data

RPM (Revolutions Per Minute)

- **How it works:** Measures how fast the engine's crankshaft is spinning. A **Crankshaft Position Sensor** (usually magnetic or optical) counts the rotations by tracking teeth on a spinning wheel.
- **How it's collected:** This data is constantly broadcast over the CAN bus by the Engine Control Unit (ECU) and can be pulled instantly via the OBD-II port.
- 

Gear

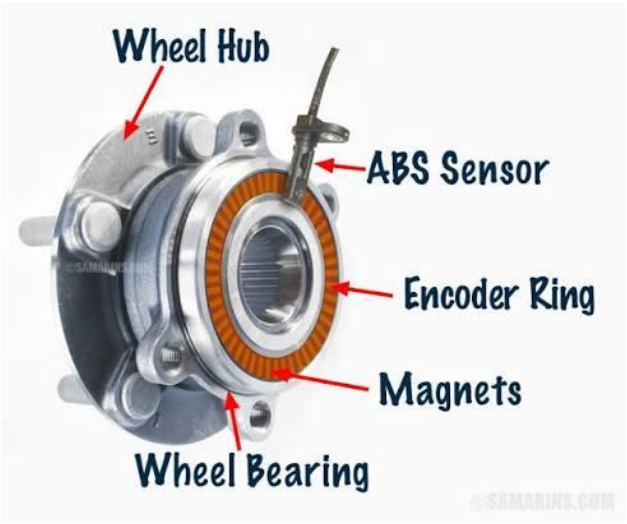
- **How it works:** In automatic transmissions, the Transmission Control Module (TCM) uses electronic switches or fluid pressure sensors to know exactly what gear is

engaged. In manual transmissions, the gear position is often calculated mathematically by comparing the current RPM against the current vehicle Speed.

- **How it's collected:** Pulled directly from the TCM or ECU over the CAN bus.

Speed

- **How it works: Wheel Speed Sensors** (part of the ABS/Anti-lock Braking System) measure how fast the tires are spinning. Alternatively, a vehicle speed sensor (VSS) on the transmission output shaft measures drivetrain speed.
- **How it's collected:** Broadcasted by the ABS/ECU to the CAN bus. (Note: Telematics units often cross-reference this with GPS speed for accuracy).



2. Fuel & Climate Efficiency

Fuel Mileage (Fuel Economy)

- **How it works:** Vehicles don't actually have a physical "fuel flow meter." Instead, the ECU knows exactly how long the fuel injectors stay open (pulse width) and the fuel pressure. By calculating how much fuel is sprayed per millisecond and comparing it to the distance traveled, it calculates real-time and average fuel mileage.
- **How it's collected:** Calculated by the ECU and read as an OBD parameter (Mass Air Flow + Fuel Trim data can also be used to calculate this externally if needed).

AC Running

- **How it works:** When a driver turns on the Air Conditioning, a signal is sent to the ECU to engage the **AC compressor clutch** (or control a variable displacement compressor). Because the AC puts a heavy mechanical load on the engine, the ECU needs to know it's on so it can slightly increase idle RPM to prevent stalling.
- **How it's collected:** Captured as a binary status (ON/OFF or 1/0) via a specific PID (Parameter ID) from the body control module or engine ECU.

3. Driver Controls & Inputs

Brake Pedal

- **How it works:** At minimum, a simple **Brake Switch** near the pedal detects if the brake is pressed (ON/OFF) to light up the brake lights. In modern vehicles, a **Brake Pressure Sensor** inside the hydraulic braking lines measures exactly how hard the driver is pressing the pedal (measured in PSI or Bar).
- **How it's collected:** ON/OFF status is readily available on the CAN bus. Precise pressure data is usually read from the ABS/ESC (Electronic Stability Control) module.

Clutch Pressure in %

- **How it works:** Manual transmission vehicles use a **Clutch Pedal Position (CPP) sensor**. This is usually a potentiometer or a magnetic Hall-effect sensor attached to the pedal assembly. It measures the physical travel of the pedal from 0% (fully released) to 100% (fully depressed).
- **How it's collected:** Read from the ECU/Transmission data streams. *Note: On heavy commercial trucks or automated manuals, a hydraulic sensor might measure actual*

fluid pressure in the clutch actuator line.

4. Tire & Driver Interaction

Tire OBD Data & TPMS (Tire Pressure Monitoring System)

- **How it works:** There are two types of TPMS:
    - **Direct TPMS (Most Common for Data):** Physical, battery-powered sensors are mounted inside each tire valve stem. They measure actual pressure and temperature and transmit this data via **Radio Frequency (RF)** (usually 315MHz or 433MHz) to a receiver in the car.
    - **Indirect TPMS:** Uses the ABS wheel speed sensors. If a tire loses pressure, its diameter shrinks slightly, causing it to spin faster than the other tires. It infers a flat but doesn't give actual pressure numbers.
- **How it's collected:** The car's TPMS receiver decodes the RF signals and pushes the pressure/temperature data onto the CAN bus. You can extract individual tire through

specific OBD-II PIDs.

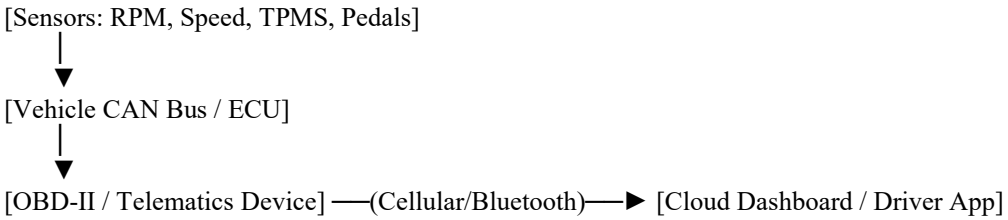
Communication with Drivers

- **How it works:** This is the bridge between the data collected and human behavior.
    - **In-Cabin Hardware:** Can be a physical screen, a set of LED lights, or audio buzzers connected to a telematics box.
    - **Software Layer:** If a driver exceeds the speed limit, harsh brakes, or idles too long with the AC on, the telematics device processes this rule and triggers an alert.

- **How it works in practice:** It can provide **real-time feedback** (e.g., an audible beep when speeding so the driver slows down immediately) or **delayed feedback** (e.g., pushing the data to a cloud dashboard for fleet managers to score the driver's safety and fuel efficiency).

How to Collect All of This Together

To collect all these data points simultaneously, you typically use an **OBD-II Telematics Dongle** or a fleet management hardware unit wired into the vehicle's diagnostic port.



Standard OBD-II readers can easily get RPM, Speed, and Fuel Mileage using generic codes. However, manufacturer-specific data points—like *exact brake pressure, clutch percentage, and individual tire pressures*—often require reading **Proprietary/Enhanced PIDs** specific to the vehicle make and model.

ELM327: How 32 Bytes of Vehicle Data Are Structured and Sent

The Protocol Stack (ISO 15765-2)

When the ELM327 reads data from the car, it does not talk directly to the engine. It sits on top of a layered protocol stack. The layer the project lead referred to is **ISO 15765-2,** also called CANTP (CAN Transport Protocol). This is the transport layer that sits between the raw CAN bus signals and the OBD-II application layer (SAE J1979).

The full stack from bottom to top looks like this:

Layer 1 — Physical: CAN bus at 500 kbps (ISO 11898). The actual electrical signal on the two wires, CAN-H and CAN-L.

Layer 2 — Data Link: CAN frame (ISO 11898). Each raw CAN frame carries an 11-bit or 29-bit ID plus up to 8 bytes of data.

Layer 3 — Transport: ISO 15765-2 (CAN-TP). Handles segmentation when a message is larger than 8 bytes. Defines Single Frame (SF), First Frame (FF), Consecutive Frame (CF), and Flow Control (FC) message types.

Layer 4 — Application: SAE J1979 / ISO 15031 (OBD-II PIDs). This is what the ELM327 exposes via AT commands — Mode 01 for live data, Mode 02 for freeze frame, Mode 03 for fault codes, etc.

In simple terms: the ELM327 sends a PID request (e.g., “01 0C” for RPM) down through this stack as a CAN frame, the ECU responds, and the ELM327 decodes the response back into human-readable hex and hands it to the ESP32 over UART.

The 32-Byte Telemetry Packet

Raw OBD-II responses return one parameter at a time, each in its own CAN frame. For this project, the ESP32 collects individual PID responses and packs them into a single **32-byte binary payload** before forwarding to the backend. This is far more efficient than sending JSON strings over a limited radio link. The structure below is the agreed packet layout:

Bytes	Parameter	Type / Range	Notes
0–1	Engine RPM	uint16 (0–16383 RPM)	OBD PID 0x0C. Raw ECU value ÷ 4 = RPM.
2	Vehicle Speed	uint8 (0–255 km/h)	OBD PID 0x0D.
3	Coolant Temp	uint8 (−40 to 215°C)	OBD PID 0x05. Stored as value + 40 offset.
4	Engine Load	uint8 (0–100%)	OBD PID 0x04. Raw ÷ 2.55 = %.
5	Throttle Position	uint8 (0–100%)	OBD PID 0x11.
6	Fuel Level	uint8 (0–100%)	OBD PID 0x2F.
7–8	MAF Air Flow	uint16 (g/s × 100)	OBD PID 0x10. Used with fuel trim to calculate fuel economy.
9	Intake Air Temp	uint8 (−40 to 215°C)	OBD PID 0x0F. Stored with + 40 offset.
10	AC Status	uint8 (0 = OFF, 1 = ON)	From BCM/ECU. May be manufacturer-specific PID on some vehicles.
11	Brake Status	uint8 (0 = released, 1 = pressed)	From ABS/ECU over CAN bus.
12	Clutch Position	uint8 (0–100%)	0 = fully released, 100 = fully depressed.
13–16	GPS Latitude	int32 (degrees × 10^6)	e.g., 12.920300 stored as 12920300. From NEO-6M.
17–20	GPS Longitude	int32 (degrees × 10^6)	e.g., 80.131600 stored as 80131600. From NEO-6M.
21–22	GPS Speed	uint16 (km/h × 10)	Cross-reference with OBD speed for accuracy.
23–26	UTC Timestamp	uint32 (Unix seconds)	From GPS time. Ties every packet to an exact moment.
27	DTC Count	uint8 (0–255)	Number of active fault codes. OBD Mode 03.
28–29	Battery Voltage	uint16 (mV, e.g., 12400)	12V healthy, below 11.8V needs attention.
30	GPS Satellites	uint8	Quality indicator — useful for backend to flag low-accuracy packets.

Bytes	Parameter	Type / Range	Notes
31	Checksum (XOR)	uint8	XOR of bytes 0–30. Lets the backend reject corrupted packets.

Total: exactly 32 bytes per vehicle telemetry packet. The GPS coordinates in bytes 13–20 come from Board 2 (NEO-6M), the rest come from Board 1 (ELM327 via OBD-II CAN). Both boards ultimately push their data to the same backend.

TPMS: How 16 Bytes of Tyre Data Are Structured and Sent

Why 16 Bytes

A car has 4 tyres. Each tyre needs 4 bytes of data — 2 for pressure and 2 for temperature. 4 tyres × 4 bytes = 16 bytes exactly. This is why the project lead specified 16 bytes as the constraint: it is the minimum complete representation of all tyre states with no wasted space.

The 16-Byte TPMS Packet Structure

Bytes	Parameter	Type / Resolution	Notes
0–1	Front-Left Pressure	uint16 (kPa × 10)	e.g., 220.0 kPa stored as 2200. Normal range: 200–250 kPa.
2–3	Front-Left Temperature	int16 (°C × 10)	e.g., 38.5°C stored as 385. High-speed alert above 90°C.
4–5	Front-Right Pressure	uint16 (kPa × 10)	Same encoding as bytes 0–1.
6–7	Front-Right Temperature	int16 (°C × 10)	Same encoding as bytes 2–3.
8–9	Rear-Left Pressure	uint16 (kPa × 10)	Same encoding as bytes 0–1.
10–11	Rear-Left Temperature	int16 (°C × 10)	Same encoding as bytes 2–3.
12–13	Rear-Right Pressure	uint16 (kPa × 10)	Same encoding as bytes 0–1.
14–15	Rear-Right Temperature	int16 (°C × 10)	Same encoding as bytes 2–3.

Total: exactly 16 bytes per TPMS snapshot. The TPMS sensors broadcast over 433 MHz RF. A dedicated receiver module on Board 1 picks up this signal and the ESP32 decodes and packs it into this structure before sending it to the backend alongside the 32-byte vehicle telemetry packet.

How TPMS Sends Its Data (The RF Path)

Each physical tyre sensor contains a small battery, a pressure sensor, a temperature sensor, and a 433 MHz radio transmitter. It wakes up and broadcasts a packet roughly every 60 seconds while stationary, and more frequently (every 5–20 seconds) when the vehicle is moving above about 25 km/h. The packet contains the sensor's unique ID, pressure, temperature, and a battery-low flag. The ESP32 on Board 1 receives this via the 433 MHz RF

module, identifies which tyre it belongs to by matching the sensor ID, and places the values in the correct byte positions in the 16-byte packet above.

End Goal: Autonomous Vehicle Control via CAN

The full vision for this internship project is to eventually connect both boards to a real vehicle (a small car or an auto-rickshaw) and use the CAN bus not just to *read* data but to *write* control commands that the vehicle's ECU responds to: accelerate, decelerate, and brake.

How CAN-Based Vehicle Control Works

Reading (what we are doing now): The ESP32 sends OBD-II PID requests and the ECU responds with data. This is passive — we observe, we do not influence the vehicle.

Writing / Control (the end goal): The MCP2515 on Board 1 transmits specific CAN frame IDs that the ECU or actuator control modules are already listening for. Throttle, braking, and steering commands are sent as CAN messages with specific IDs and data payloads. The vehicle reacts as if the driver pressed the pedal or brake.

This is the same principle used in modern ADAS (Advanced Driver Assistance Systems) and is how Tesla Autopilot, cruise control, and automatic emergency braking physically work at the hardware level.

Key Steps to Get There

- Complete hardware validation: GPS confirmed, CAN loopback confirmed, ELM327 and TPMS are the immediate next steps as directed by the project lead.
- Get live OBD-II data from an actual vehicle using the ELM327. This proves the full read path works end-to-end.
- Study the vehicle-specific CAN frame IDs for the target car/auto. These are different for every make and model and are found by sniffing the CAN bus while a human manually presses the accelerator and brakes, then identifying which frame IDs change.
- Implement a CAN write loop on the ESP32 using MCP2515 that sends throttle/brake commands. Test first in a safe controlled environment (engine running, stationary) before attempting any in-motion tests.
- Integrate TPMS data into the control loop: if tyre pressure or temperature exceeds safe thresholds, the system sends a deceleration command before the driver notices a problem.